

OOP: Programación Orientada a Objetos

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas, M Victoria Tolosa

Introducción

Al principio de la materia hablamos que nuestro enfoque sería el paradigma imperativo de la programación.

Veremos ahora muy por encima los principios del paradigma de la programación orientada a objetos.

Básicamente se trata de “pensar” distinto a la hora de programar. Aunque usaremos la misma herramienta (python language).

Aplicar POO en Pytho gira en torno a utilizar **class**

SOBRE CLASES Y OBJETOS

Pensemos

- Hasta ahora en la materia vimos tipos de datos (string, integer, bool, etc).
- Así como también estructuras de datos (dict, list, set, etc).
- Si `m` es un **diccionario**, tiene sentido decir `m.append(4)`
- Si `L` es una lista, que arrojará decir `L['key']`?

Pensemos

- Recapitulando, las estructuras de datos son TADS, osea, tipos de datos abstractos.
- Alguien, en algún momento, codeó lo que significa una lista. Ese alguien codeó también lo que significa append, y dijo que append sólo aplica a listas y no a diccionarios.
- Todo esto fue codeado en C y es de código abierto:
<https://github.com/python/cpython>

Definamos

- Teniendo en cuenta todo esto, se puede decir que una lista o un diccionario son **clases**.
- Las acciones concretas asociadas a estas clases (append para listas, add para sets, etc) se las llama **métodos**.
- Cuando creamos instancias puntuales de una clase, lo que estamos creando se llama **objeto**.
- Ejemplo: si decimos $L=[1,2,3]$, L es un objeto de la clase lista



Clase



Objeto

Nosotros podemos definir clases

- Todo lo que explicamos en la filmína pasada es endémico de Python. Cuando uno instala python, ya existen las listas/diccionarios/sets.
- Con la palabra clave **class** nosotros podemos definir un nuevo tipo de dato (clase) y al mismo asignarle dentro variables (**atributos**), y funciones (**métodos**).

Implementación

- **class** <nombre>: Esta es la manera de crear una clase, el nombre por convención se espera que lleve mayúscula.
- **def __init__(self, atributo_1, atributo_2, ...)**: Esta es la función init. Toda clase debe tener una. Los atributos que se definen en el corolario de esta function son los que si o si deben pasarse a la hora de crear una function.
- **self**: Esta palabra clave entra a jugar cuando tratamos con clases. Traducido significa “el objeto que estamos creando a la hora de implementar esta clase”.

Implementación

- Quitando `__init__` todo el resto de las **functions** que definamos dentro de una class son los **métodos** de la misma.
- **self.atributo** es la manera de referenciar un atributo de la clase generada

Ejemplo

```
class Cuadrado:

    def __init__(self,lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self,cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False
```

- Qué es Cuadrado?
- Qué son perimetro/lado?
- Qué son (y que hacen) shrink y pasa_test?

Ejemplo

#Paso a paso, que estamos haciendo en cada secuencia?

```
cuadra=Cuadrado(5)
cuadra.perimetro
cuadra.lado
cuadra.calcula_perimetro()
cuadra.perimetro
```

```
cuadra.shrink(4.5)
cuadra.lado
cuadra.calcula_perimetro()
cuadra.perimetro
pasa=cuadra.pasa_test()
```

Ejercicio

Observar el siguiente script, explicar (comentando) que es lo que hace el método **resto_sup**

Métodos Analíticos y Programación

```
import math as mt
# Palabra clave class + nombre de la clase que estás creando
class Cuadrado:

    # Inicializador, si o si debe tenerlo.
    # El input de la función init es lo que obligatoriamente debes pasarle a la class
    # La palabra clara "self" refiera al objeto que estás creando al ejecutar esta clase
    def __init__(self, lado):
        self.lado=lado
        self.perimetro=4*lado
        self.sup=lado**2

    #Aca estoy creando un método que resta la superficie de un cuadrado menor a nuestro
    #cuadrado ya creado y con la sup restante crea un cuadrado más chico.
    #Redefiniendo el lado de nuestro cuadrado
    def resto_sup(self, lado_comparativo):
        if lado_comparativo>self.lado:
            return 'No se puede restar una sup. más grande que la sup. original'
        self.sup= self.sup-(lado_comparativo**2)
        self.lado=mt.sqrt(self.sup)
        self.perimetro=self.lado*4

    #Método para saber si el cuadrado resultante tiene una superficie "aceptable":
    #Decimos que aceptable es mayor a 4
    def es_aceptable(self):
        if self.sup>1:
            return True
        else:
            return False
```

Ejercicio_2

Crear una class llamada Estadio que tome como inputs:

- el largo de la cancha
- el ancho de la misma
- la categoría del club al que pertenece

Un método que calcule la capacidad del estadio, siendo la capacidad de una grada $L * n_{\text{filas}}$

Si el club es de la B, las gradas tienen 100 filas, si es de primera tiene 200. Todos los estadios tienen 4 gradas.

Otro método que reciba la cantidad de gente en un partido y retornee si se está excediendo o no la capacidad

REPRESENTACIÓN Y COMPARACIÓN

Representación

¿Qué sucede si ejecutamos un print al objeto cuadrado que creamos en las filminas anteriores?

¿Qué vendría a ser esto?

```
<__main__.Cuadrado object at 0x722b4a98d070>
```

```
class Cuadrado:

    def __init__(self, lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self, cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

cuad=Cuadrado(4)
print(cuad)
```

Representación

El objeto “cuadrado” es nuevo para python, no sabe como printearlo. Si le pedimos que lo printee, lo que hará será printear su placeholder en memoria.

Nosotros debiéramos usar el método `__repr__` para indicar que se printea cuando printeamos nuestro objeto.

Representación

```
class Cuadrado:

    def __init__(self,lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self,cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

    def __repr__(self):
        return '<'+str(self.lado)+'cm de lado'+>'

cuad=Cuadrado(4)
print(cuad)
```

Ahora printea esto:

```
<4cm de lado>
```

Importante: el output del método `__repr__` debe ser siempre un string. Si no arroja error

Comparación

Algo similar sucede con la comparación. ¿Qué sucede si creamos 2 cuadrados, `cuad1` y `cuad2`. Decimos que `cuad1` tiene lado 4 y `cuad2` tiene lado 5.

Acto seguido planteamos `cuad2 > cuad1`. La lógica dice que debe ser `True`, no?

Bueno, **no**, otra vez, python no sabe la naturaleza de la clase que acabamos de inventar. Si en lugar de tener un solo input (lado) tuviésemos 2 (lado y fecha de creación). ¿Qué cuadrado sería el más "grande" para python? ¿El más viejo o el de lado más largo?

Comparación

Esto se soluciona con una serie de métodos que se pueden agregar a la clase. Como son:

- ▶ `__lt__(self, other):` `objeto_1 < objeto_2`
- ▶ `__le__(self, other):` `objeto_1 <= objeto_2`
- ▶ `__ne__(self, other):` `objeto_1 != objeto_2`
- ▶ `__gt__(self, other):` `objeto_1 > objeto_2`
- ▶ `__ge__(self, other):` `objeto_1 >= objeto_2`

Comparación –Ejemplo

```
class Cuadrado:

    def __init__(self,lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self,cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

cuad1=Cuadrado(4)
cuad2=Cuadrado(4)

print(cuad1==cuad2)
```

Esto va a printear False, ya que **lo que estás comparando son espacios en memoria**. cuad1 y cuad2 son 2 objetos distintos que ocupan un slot distinto en la memoria.

A nosotros nos gustaría que, dado 2 cuadrados de igual lado, python nos dijera que son iguales ...

Comparación –Ejemplo

```
class Cuadrado:

    def __init__(self,lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self,cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

    def __eq__(self, other):
        return self.lado == other.lado

cuad1=Cuadrado(4)
cuad2=Cuadrado(4)
cuad3=Cuadrado(5)
print(cuad1==cuad2)
print(cuad1==cuad3)
```

Bueno ahora si, gracias a la función **__eq__** ahora printea True cuando se lo compara con un cuadrado de igual tamaño y False cuando se lo compara con uno más chico/grande.

Comparación –Ejemplo2

```
class Cuadrado:

    def __init__(self, lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self, cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

    def __eq__(self, other):
        return self.lado == other.lado

cuad1=Cuadrado(4)
cuad2=Cuadrado(4)
cuad3=Cuadrado(5)
print(cuad1>cuad2)
```

Esto así como está arroja un error.
Ya que la comparación entre
objetos "Cuadrado" no está
definida

Comparación –Ejemplo2

```
class Cuadrado:
    def __init__(self,lado):
        self.lado=lado

    def calcula_perimetro(self):
        self.perimetro=4*self.lado

    def shrink(self,cuanto):
        self.lado=self.lado-cuanto

    def pasa_test(self):
        if self.lado>1:
            return True
        else:
            return False

    def __eq__(self, other):
        return self.lado == other.lado

    def __lt__( self , other ):
        return self.lado < other.lado

cuad1=Cuadrado(4)
cuad2=Cuadrado(4)
cuad3=Cuadrado(5)
print(cuad1>cuad2)
print(cuad2<cuad3)
```

En esta nueva situación,
returneamos primero False y
después True

Ejercicio_3

Retomemos el ejemplo del estadio.

Modificar la clase de tal manera de que se juzgue un estadio como más chico o más grande que otro **por la superficie de su campo de juego**. Independientemente de su capacidad